

Отчет по домашнему заданию

Интеграция ClickHouse с PostgreSQL

Цель	Настроить базовую интеграцию ClickHouse с PostgreSQL с использованием функции postgresql, движка таблицы PostgreSQL и движка базы данных PostgreSQL.
Среда	Docker Compose: PostgreSQL 16 + ClickHouse Server.
Датасет	Готовый публичный CSV-датасет Telco Customer Churn, загруженный в PostgreSQL через COPY.
База PostgreSQL	test_db
Таблица PostgreSQL	public.telco_customer_churn
Количество строк	7043

1. Развертывание окружения в контейнерах

Для выполнения задания были развернуты два контейнера: PostgreSQL для хранения датасета и ClickHouse для чтения данных из PostgreSQL. Контейнеры находятся в одной сети Docker Compose, поэтому ClickHouse обращается к PostgreSQL по имени сервиса postgres:5432.

```
services:
  postgres:
    image: postgres:16
    container_name: hw_postgres
    environment:
      POSTGRES_DB: test_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres_password
    ports:
      - "5432:5432"
    volumes:
      - ./postgres-init:/docker-entrypoint-initdb.d
      - postgres_data:/var/lib/postgresql/data

  clickhouse:
    image: clickhouse/clickhouse-server:latest
    container_name: hw_clickhouse
    depends_on:
      - postgres
    ports:
      - "8123:8123"
      - "9000:9000"
    ulimits:
      nofile:
        soft: 262144
        hard: 262144

volumes:
  postgres_data:

docker compose down -v
docker compose up -d
```

2. Загрузка готового датасета в PostgreSQL

В PostgreSQL создана таблица public.telco_customer_churn. CSV-файл был загружен на этапе инициализации контейнера командой COPY.

```
CREATE TABLE public.telco_customer_churn (
  customer_id VARCHAR(50),
  gender VARCHAR(20),
  senior_citizen INT,
  partner VARCHAR(10),
  dependents VARCHAR(10),
  tenure INT,
  phone_service VARCHAR(10),
  multiple_lines VARCHAR(50),
  internet_service VARCHAR(50),
  online_security VARCHAR(50),
  online_backup VARCHAR(50),
  device_protection VARCHAR(50),
  tech_support VARCHAR(50),
  streaming_tv VARCHAR(50),
```

```

streaming_movies VARCHAR(50),
contract VARCHAR(50),
paperless_billing VARCHAR(10),
payment_method VARCHAR(100),
monthly_charges NUMERIC(10, 2),
total_charges VARCHAR(50),
churn VARCHAR(10)
);

COPY public.telco_customer_churn (
customer_id, gender, senior_citizen, partner, dependents, tenure,
phone_service, multiple_lines, internet_service, online_security,
online_backup, device_protection, tech_support, streaming_tv,
streaming_movies, contract, paperless_billing, payment_method,
monthly_charges, total_charges, churn
)
FROM '/docker-entrypoint-initdb.d/telco_customer_churn.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ',', QUOTE '');

SELECT count(*) FROM public.telco_customer_churn;
SELECT * FROM public.telco_customer_churn LIMIT 5;

```

```

test_db=# SELECT count(*) FROM public.telco_customer_churn;

SELECT *
FROM public.telco_customer_churn
LIMIT 5;
count
-----
 7043
(1 row)

```

Проверка загрузки датасета в PostgreSQL: 7043 строки.

3. Чтение данных из PostgreSQL через функцию postgresql в ClickHouse

На стороне ClickHouse выполнен запрос к таблице PostgreSQL с использованием table function postgresql(...). Результат подтвердил доступность данных из PostgreSQL.

```

SELECT count()
FROM postgresql(
    'postgres:5432',
    'test_db',
    'telco_customer_churn',
    'postgres',
    'postgres_password',
    'public'
);

```

Результат:
count() = 7043

```

c6606ff7b764 :) SELECT count()
FROM postgresql(
    'postgres:5432',
    'test_db',
    'telco_customer_churn',
    'postgres',
    'postgres_password',
    'public'
);

SELECT count()
FROM postgresql('postgres:5432', 'test_db', 'telco_customer_churn', 'postgres', 'postgres_password', 'publ
ic')

Query id: bedec016-920c-4060-b0c9-6e165f2b9080

1. count()
   7043

1 row in set. Elapsed: 0.014 sec. Processed 7.04 thousand rows, 35.22 KB (493.61 thousand rows/s., 2.47 MB
/s.)
Peak memory usage: 245.59 KiB.

```

Результат запроса ClickHouse через функцию postgresql(...): 7043 строки.

4. Таблица ClickHouse через движок PostgreSQL

Создана отдельная база ClickHouse pg_integration и таблица telco_customer_churn_pg, которая ссылается на таблицу PostgreSQL через движок PostgreSQL.

```
CREATE DATABASE IF NOT EXISTS pg_integration;
CREATE TABLE pg_integration.telco_customer_churn_pg
(
    customer_id String,
    gender String,
    senior_citizen Int32,
    partner String,
    dependents String,
    tenure Int32,
    phone_service String,
    multiple_lines String,
    internet_service String,
    online_security String,
    online_backup String,
    device_protection String,
    tech_support String,
    streaming_tv String,
    streaming_movies String,
    contract String,
    paperless_billing String,
    payment_method String,
    monthly_charges Decimal(10, 2),
    total_charges String,
    churn String
)
ENGINE = PostgreSQL(
    'postgres:5432',
    'test_db',
    'telco_customer_churn',
    'postgres',
    'postgres_password',
    'public'
);
SELECT
    count() AS rows_count,
    avg(monthly_charges) AS avg_monthly_charges
FROM pg_integration.telco_customer_churn_pg;

Результат:
rows_count = 7043
avg_monthly_charges = 64.76169246059918
```

```
c6606ff7b764 :) SELECT
    count() AS rows_count,
    avg(monthly_charges) AS avg_monthly_charges
FROM pg_integration.telco_customer_churn_pg;

SELECT
    count() AS rows_count,
    avg(monthly_charges) AS avg_monthly_charges
FROM pg_integration.telco_customer_churn_pg

Query id: 1dc96b59-f310-41cd-af6a-97d9ade6e2e8

1. ┌-rows_count-┐ ┌-avg_monthly_charges-┐
   │      7043 │ │      64.76169246059918 │
   └──────────┘ └──────────────────┘

1 row in set. Elapsed: 0.018 sec. Processed 7.04 thousand rows, 56.34 KB (385.78 thousand rows/s., 3.09 MB /s.)
Peak memory usage: 193.74 KiB.
```

Проверка таблицы ClickHouse с движком PostgreSQL.

5. База данных ClickHouse через движок PostgreSQL

Также создана база данных ClickHouse с движком PostgreSQL. Она отображает таблицы из схемы public базы PostgreSQL test_db.

```
CREATE DATABASE IF NOT EXISTS postgres_db_engine
ENGINE = PostgreSQL(
    'postgres:5432',
    'test_db',
    'postgres',
    'postgres_password',
    'public'
);
SHOW TABLES FROM postgres_db_engine;

SELECT count()
```

```
FROM postgres_db_engine.telco_customer_churn;
```

Результат:

```
SHOW TABLES: telco_customer_churn
```

```
count() = 7043
```

```
SHOW TABLES FROM postgres_db_engine
```

```
Query id: 9fa59137-1715-478e-9214-11f729cffe1a
```

```
1. | name  
   | telco_customer_churn
```

```
1 row in set. Elapsed: 0.019 sec.
```

```
c6606ff7b764 :) SELECT count()  
FROM postgres_db_engine.telco_customer_churn;
```

```
SELECT count()  
FROM postgres_db_engine.telco_customer_churn
```

```
Query id: e58cf5f8-f714-4de0-a28c-8066c3eb10f9
```

```
1. | count()  
   | 7043
```

```
1 row in set. Elapsed: 0.009 sec. Processed 7.04 thousand rows, 35.22 KB (794.28 thousand rows/s., 3.97 MB /s.)  
Peak memory usage: 171.09 KiB.
```

Проверка базы данных ClickHouse с движком PostgreSQL.

6. Итог

Интеграция ClickHouse с PostgreSQL выполнена успешно. Датасет загружен в PostgreSQL, после чего данные были доступны из ClickHouse тремя способами: через функцию `postgresql(...)`, через таблицу с движком PostgreSQL и через базу данных с движком PostgreSQL. Во всех проверках получено одинаковое количество строк - 7043.